

EXP NO: 5	CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION WITH PCA
------------------	--

AIM:

To demonstrate the application of Unsupervised Learning models, specifically K-Means clustering for grouping data points and Principal Component Analysis (PCA) for dimensionality reduction and visualization, using a suitable dataset.

Code 1:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# =====
# PART 1: K-MEANS CLUSTERING
# =====

# Generate synthetic dataset
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.60, random_state=42)
df_kmeans = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])

# Flattened Elbow Method (No multi-line for-loop)
wcss = [KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=42).fit(X).inertia_ for i in range(1, 11)]

# Output 1: Elbow Method Plot
plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()

```

```

# Apply K-Means with Optimal K
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, n_init=10, random_state=42)
clusters = kmeans.fit_predict(X)

# Output 2: Silhouette Score for K-Means
print("Silhouette Score for K-Means:", silhouette_score(X, clusters))

# Output 3: K-Means Cluster Visualization
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', s=50, alpha=0.8)
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering Result')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()

# =====
# PART 2: PCA & PCA + K-MEANS
# =====

# Generate 4D dataset for PCA
X_pca, y_pca = make_blobs(n_samples=500, n_features=4, centers=4, cluster_std=1.0, random_state=25)
scaler = StandardScaler()
X_pca_scaled = scaler.fit_transform(X_pca)

# Reduce dimensions from 4D to 2D
pca = PCA(n_components=2)
principal_components = pca.fit_transform(X_pca_scaled)

```

```

# Output 4: PCA Explained Variance Ratio
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Output 5: Cumulative Explained Variance Plot
plt.figure(figsize=(6, 4))
plt.bar(range(1, 3), pca.explained_variance_ratio_, alpha=0.7, align='center', label='Individual Variance')
plt.step(range(1, 3), np.cumsum(pca.explained_variance_ratio_), where='mid', color='red', label='Cumulative Variance')
plt.ylabel('Explained Variance Ratio')
plt.xlabel('Principal Components')
plt.title('PCA Explained Variance')
plt.legend(loc='best')
plt.show()

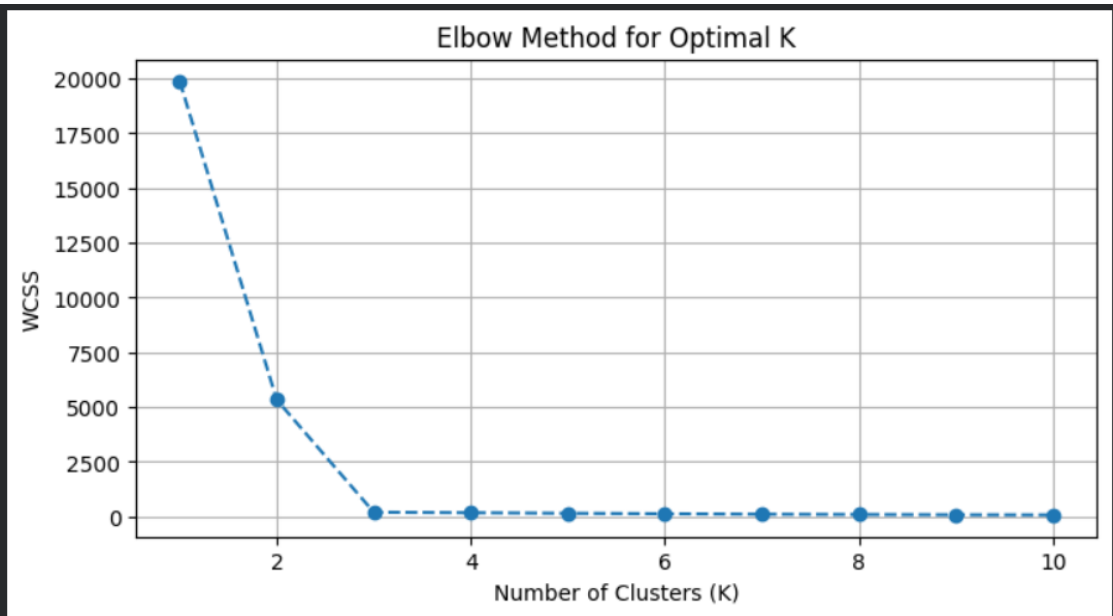
# Fit K-Means on PCA-transformed data
kmeans_pca = KMeans(n_clusters=4, init='k-means++', max_iter=300, n_init=10, random_state=42)
clusters_pca = kmeans_pca.fit_predict(principal_components)

# Output 6: Silhouette Score on PCA Data
print("Silhouette Score on PCA Data:", silhouette_score(principal_components, clusters_pca))

# Output 7: Clusters Visualized in 2D PCA Space
plt.figure(figsize=(8, 5))
plt.scatter(principal_components[:, 0], principal_components[:, 1], c=clusters_pca, cmap='plasma', s=50, alpha=0.8)
plt.scatter(kmeans_pca.cluster_centers[:, 0], kmeans_pca.cluster_centers[:, 1], s=200, c='cyan', marker='X', label='Centroids')
plt.title('K-Means Clusters in 2D PCA Reduced Space')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()

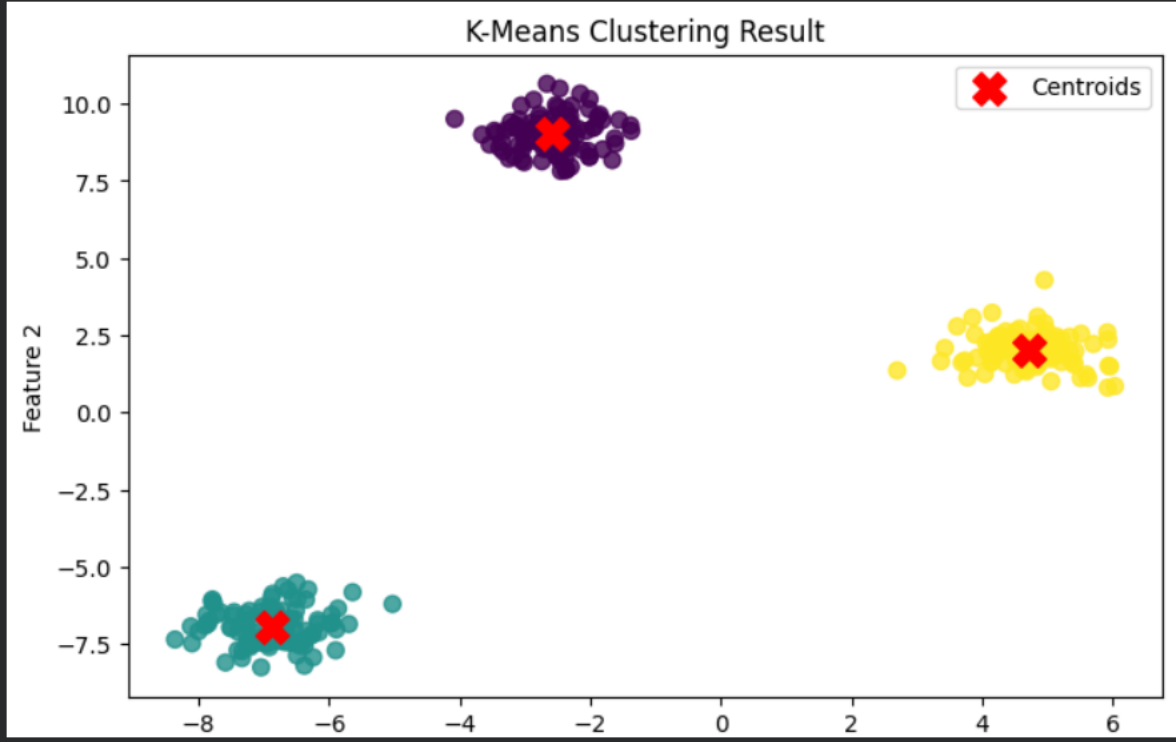
```

Output 1:

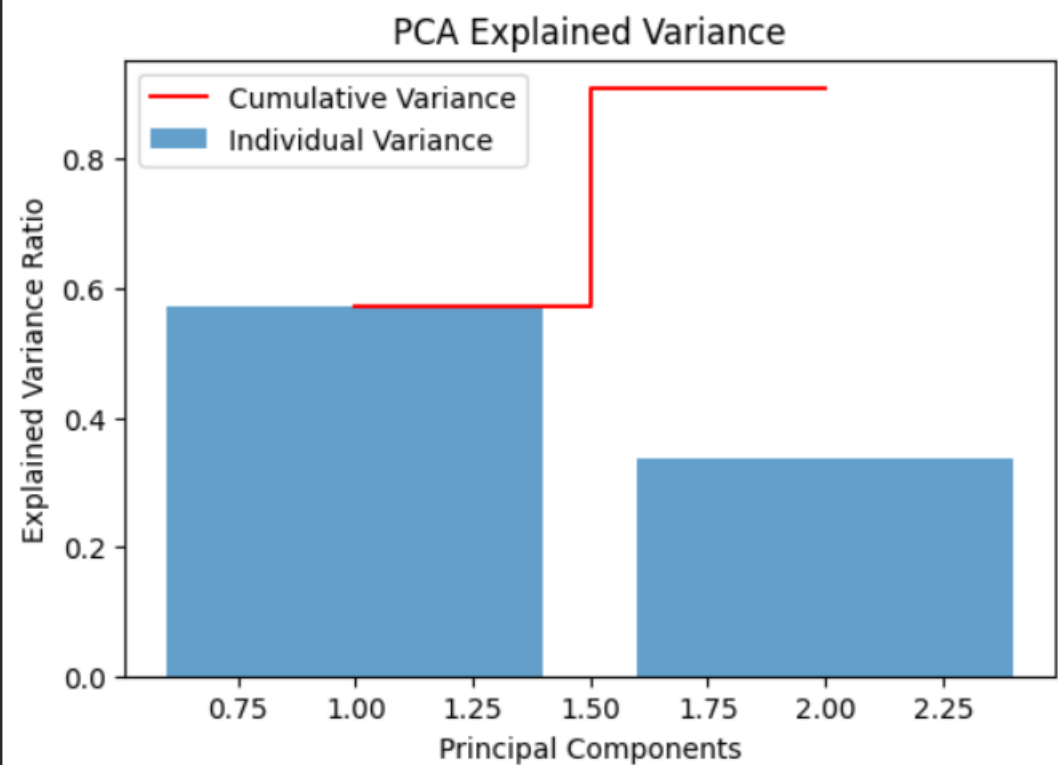


Silhouette Score for K-Means: 0.9083834454815235

Silhouette Score for K-Means: 0.9083834454815235



Explained Variance Ratio: [0.57208431 0.33622342]



Silhouette Score on PCA Data: 0.7764383951477865



CODE 2:

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Load Iris dataset
iris = load_iris()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(iris.data)

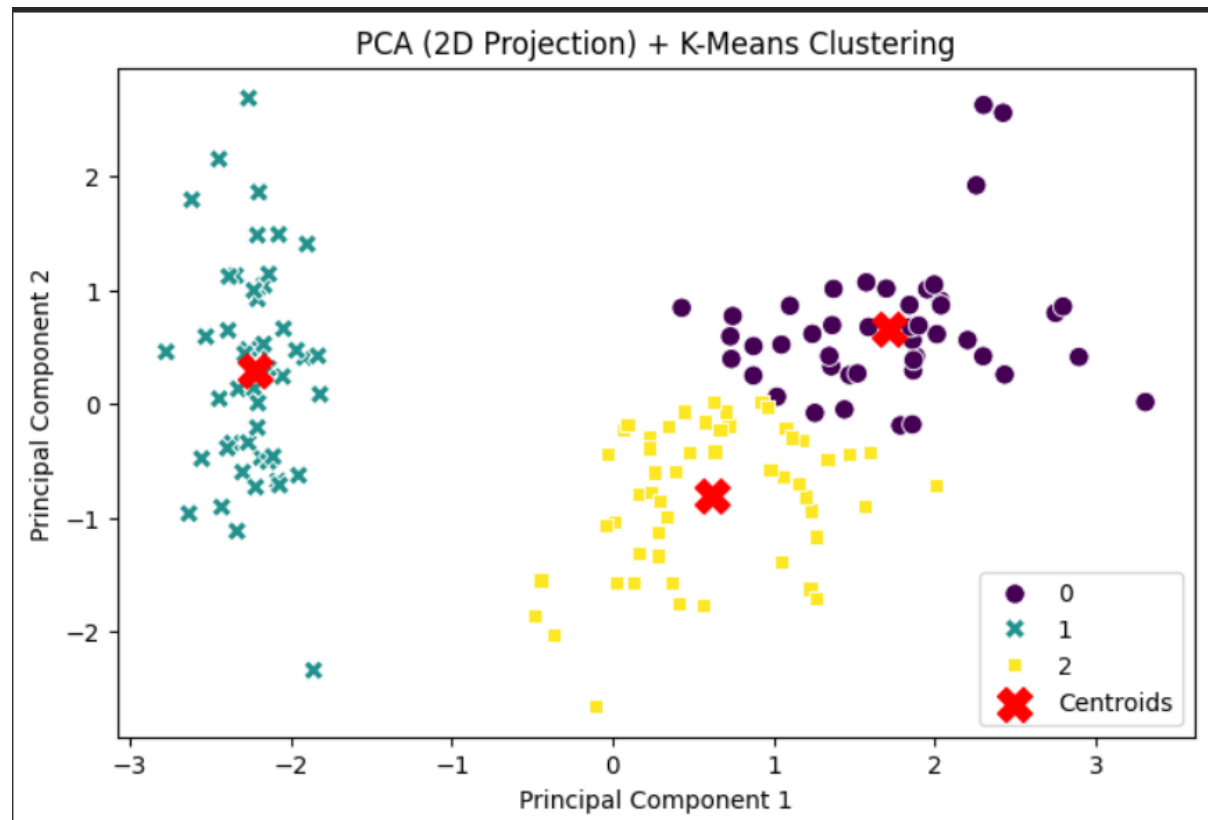
# Dimensionality Reduction with PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# K-Means Clustering
kmeans = KMeans(n_clusters=3, n_init=10, random_state=101)
labels = kmeans.fit_predict(X_pca)

# Extract principal component coordinates
pc1 = X_pca[:, 0]
pc2 = X_pca[:, 1]
centroids = kmeans.cluster_centers_

# Scatter Plot Visualization (Single-line function calls)
plt.figure(figsize=(8, 5))
sns.scatterplot(x=pc1, y=pc2, hue=labels, palette='viridis', s=70, style=labels)
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red', marker='x', label='Centroids')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA (2D Projection) + K-Means Clustering")
plt.legend()
```

Output 2:



RESULT:

The K-Means clustering and Principal Component Analysis (PCA) techniques were successfully implemented on the given dataset.

- **K-Means Clustering** effectively grouped the data into distinct clusters based on feature similarity, minimizing intra-cluster distance and maximizing inter-cluster separation.
- **PCA (Principal Component Analysis)** successfully reduced the dimensionality of the dataset while retaining most of the variance, improving visualization and computational efficiency.

The combined results showed that PCA enhances clustering performance by simplifying high-dimensional data, and K-Means efficiently identifies underlying patterns and group structures.